Narration Studio guardrails

How guardrails are implemented and where they actually run.

> **Confirmed** means the behavior is in this repository's source. **Inferred** means it comes from Microsoft Agent Framework (MAF) middleware.

Summary

Guardrails in this app are **agent middleware**, not a global security layer. They wrap every MAF specialist when `handoff` or `sequential` orchestration builds a crew. **Direct mode does not run them.** Hard production caps (exactly 8 scripts, at most 5 slides) are enforced in tools by throwing, not by `ProductionGuardrails`.

There are three related mechanisms:

1. **`ProductionGuardrails`** — MAF middleware on each agent (`RunAsync` + `FunctionAsync`).
2. **Prompt guardrails** — text injected by `ProductionContextProvider` ("Guardrails you must obey").
3. **Tool validation** — `ProductionTools` throws if script/slide counts are wrong.

Only (1) can refuse a model call before the LLM runs. (2) is advisory. (3) runs whether or not agents are used.

Where they are wired

In `AgentFactory.TryCreate`, one `ProductionGuardrails` instance is created and passed into every agent:

```
var guardrails = new ProductionGuardrails(_options, _loggerFactory.CreateLogger<ProductionGuardrails>
());
```

Each agent is then wrapped:

```
return agent
    .AsBuilder()
    .Use(guardrails.RunAsync, null)
    .Use(guardrails.FunctionAsync)
    .Build();
```

Source: `src/NarrationStudio/Agents/AgentFactory.cs`. **Inferred:** `.Use(RunAsync)` runs around the agent's chat turn; `.Use(FunctionAsync)` runs around each tool invocation.

`DirectPipelineRunner` never constructs or calls `ProductionGuardrails`.

RunAsync — message filter and block

File: `src/NarrationStudio/Guardrails/ProductionGuardrails.cs`.

Before the inner agent runs:

1. Every inbound `ChatMessage` is passed through `FilterMessage`.
2. If any filtered message text contains `[BLOCKED`, the inner agent is **not** called. The user gets a fixed refusal: stay on transcript segmentation, slides, TTS, and video assembly. A warning is logged: `Guardrail blocked an inbound request.`
3. Otherwise the inner agent runs on the **filtered** messages.
4. Outbound assistant messages are filtered again (redaction only; blocked terms apply only to `ChatRole.User`).

FilterMessage

Blocked terms (case-insensitive substring on **user** messages only): `harmful`, `illegal`, `violence`, `exploit`, `malware`, `bypass paywall`.

A match replaces the entire user message with `[BLOCKED: content outside production policy]`, which then triggers the short-circuit in `RunAsync`.

PII redaction (all roles):

- Emails matching `\b[\w\.-]+@[\w\.-]+\.\w+\b` → `[REDACTED-EMAIL]`
- US-style phones `\b\d{3}-\d{3}-\d{4}\b` → `[REDACTED-PHONE]`

This does **not** redact transcripts on disk, ElevenLabs TTS payloads, or Direct-mode tool inputs. It only rewrites chat messages that pass through the agent middleware.

Substring blocking is naive: a user message that contains “harmful” in an unrelated sentence is refused. A jailbreak that avoids those six strings is not.

FunctionAsync — tool wrapper

Every tool call from an agent is logged (`Tool {Name} starting` / `finished`).

If the tool name contains `"Slide"` and the string result contains the word `"slide"` more times than `MaxSlidesPerSegment * SegmentCount` (default 5 × 8 = 40), it logs:

`Slide-count guardrail trimmed an oversized tool result.`

> **Confirmed:** it does not trim or reject the result. The original tool return value is passed through unchanged. The 8-script / 5-slide caps that actually stop the pipeline are in `ProductionTools` (`EnsureScripts`, `Take(MaxSlidesPerSegment)`, `SaveNarrationScripts` count check).

Prompt “guardrails” (not middleware)

`ProductionContextProvider.ProvideAIContextAsync` injects instructions such as:

- Rewrite the raw transcript before other steps (unless stubbing already did).
- Exactly `SegmentCount` scripts from the rewritten talk, never the raw transcript.
- At most `MaxSlidesPerSegment` slides per segment.
- Exact audio/video file names.
- Use tools; do not invent file names.
- Stay on the narration-to-video task.

That is **prompt policy**. A model can ignore it. Middleware + tool throws are the enforcement layers, and even those are limited as above.

Request flow (agent mode)

```
User / workflow message
    → ProductionGuardrails.RunAsync
        → FilterMessage (block terms / redact PII)
        → if [BLOCKED] → refusal, stop
        → inner agent (LLM + tools)
            → each tool: FunctionAsync (log; optional unused slide warning)
            → ProductionTools (real caps: throw on bad counts)
        → FilterMessage on outbound messages
    → agent reply
```

Direct mode skips the first and last boxes and calls `ProductionTools` only.

What is not covered

Gap	Detail
Direct orchestration	No `ProductionGuardrails` at all
Tool arguments	`FunctionAsync` does not inspect or sanitize inputs
Slide-count “trim”	Warning only; result not modified
Transcript / TTS / files	No PII redaction
Blocked-term list	Six English substrings; easy to evade
Auth / tenancy	None; this is a local console app

Key files

File	Role
`src/NarrationStudio/Guardrails/ProductionGuardrails.cs`	Blocked terms, PII regex, tool logging, unused slide warning
`src/NarrationStudio/Agents/AgentFactory.cs`	Attaches middleware to every specialist
`src/NarrationStudio/Memory/ProductionContextProvider.cs`	Prompt-level “Guardrails you must obey”
`src/NarrationStudio/Tools/ProductionTools.cs`	Hard 8-script / 5-slide validation
`src/NarrationStudio/Orchestration/DirectPipelineRunner.cs`	No guardrail middleware